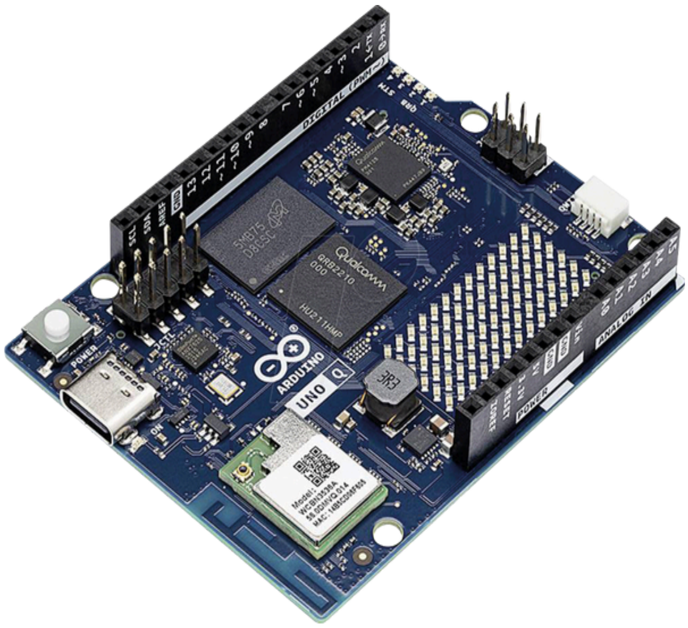
Set-up

Initial Boot & Device Provisioning

Requirements & System Components

The **Arduino UNO Q** is a unique "Dual-Brain" board combining the following:

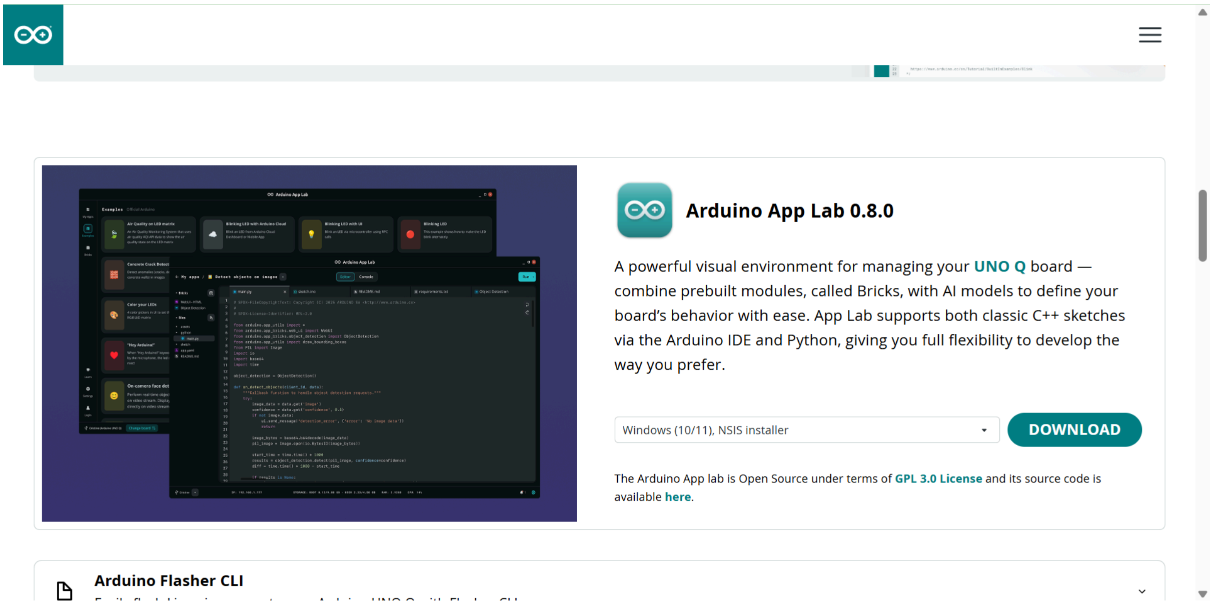
1. **Microprocessor (MPU) Side:** Qualcomm® Dragonwing™ QRB2210 running a pre-flashed Linux Debian OS. It handles high-level computing, Python scripting, and AI models (like Edge Impulse .eim files).
2. **Microcontroller (MCU) Side:** STM32U585 handles real-time control, sensors, and low-latency execution of your C++ Arduino Sketches.



Step 1 — Installing Arduino App Lab & Board Initialization

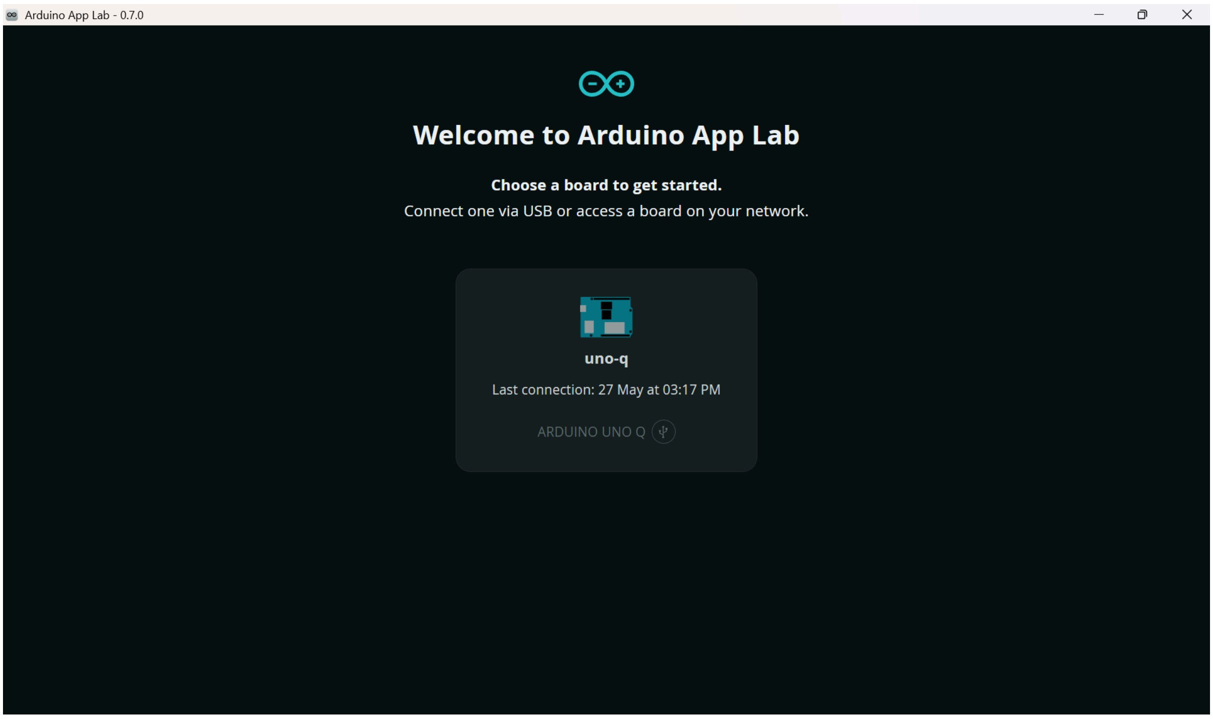
1. Download and Install the Software

- Download the desktop installer for **Arduino App Lab** matching your computer's operating system .

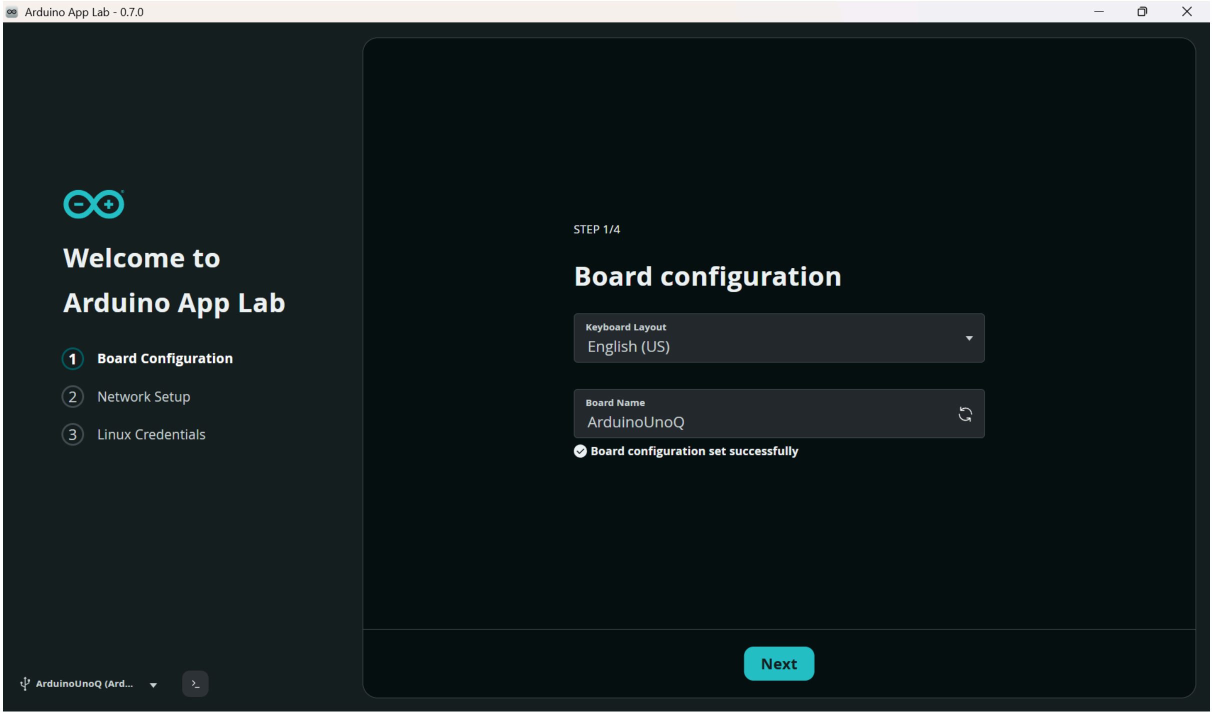


- Run the installer file and follow the on-screen prompts to complete the installation.
- Launch **Arduino App Lab** on your computer.

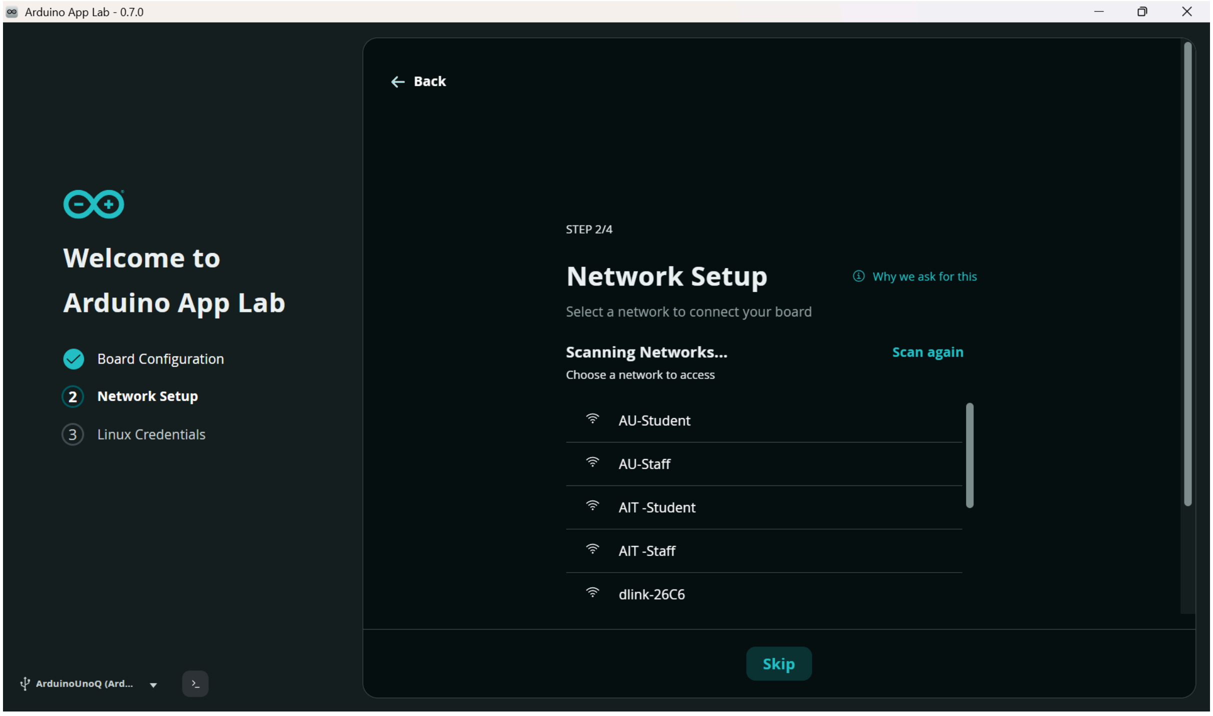
2. First-Time Wi-Fi Provisioning



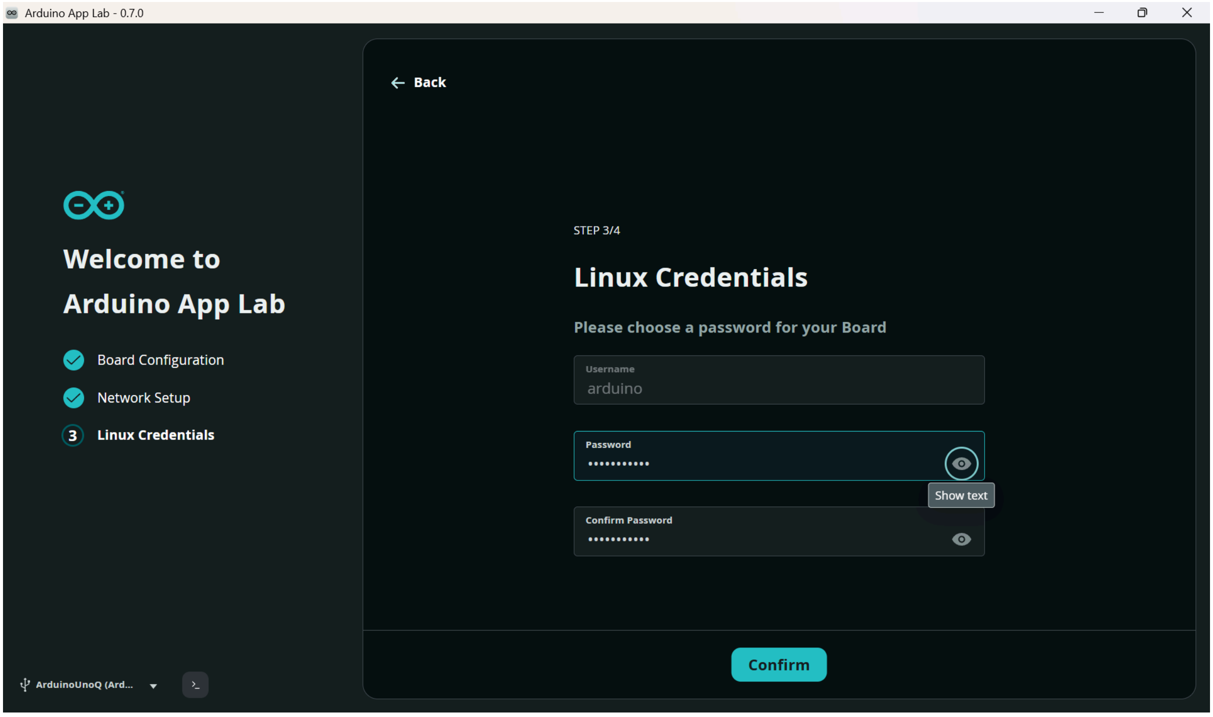
- Connect the Arduino UNO Q to a power source (like a USB-C wall adapter or power bank).
- Launch the App Lab client. Since it is the first boot, follow the on-screen provisioning wizard:
    1. **Set Passwords:** Create a secure system password for your default board account (Arduino).
    2. **Name Your Board:** Input a unique hostname for your device.



3. **Network Setup:** Select your local Wi-Fi network from the detected list, input the Wi-Fi password, and click connect.



4. Add Linux credentials:

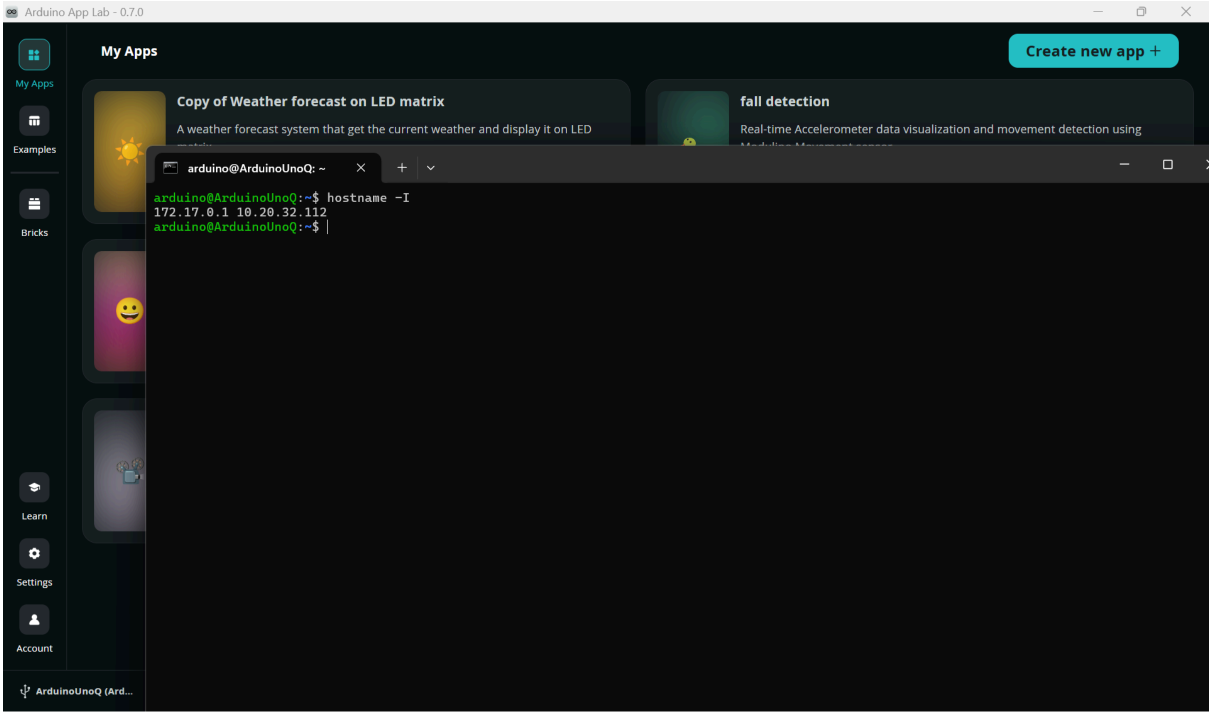


3. Locating Your Board's Unique IP Address

Once the board successfully connects to your wireless network, you must find its newly assigned IP address to establish an SSH connection. You can find it exactly here:

The App Lab Top Banner: Look at the very top menu status bar of the Arduino App Lab window. Right next to your board's custom name and a green connection status dot, the network IP address will be printed clearly inside parentheses—for example: `MyArduinoBoard (192.168.1.45)`.

The Terminal Command: Alternatively, look at the bottom half of the App Lab workspace and locate the **Terminal/Console** panel. Click inside the prompt, type `hostname -I`, and hit **Enter**. The numbers printed on the next line (e.g., `192.168.1.45`) is your board's unique Wi-Fi IP address. Copy this number down.

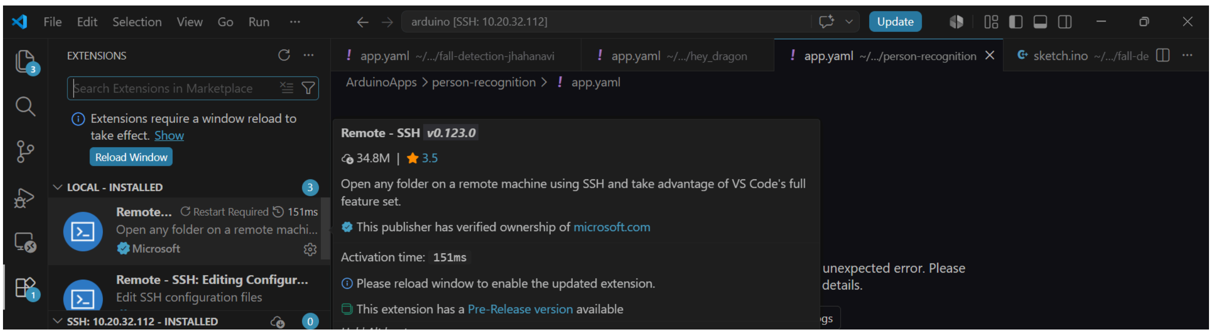


Step 2 — Setting up VS Code and Connecting via Wireless SSH

Now that the board is broadcasting on your wireless network, you can completely close the App Lab and move to VS Code.

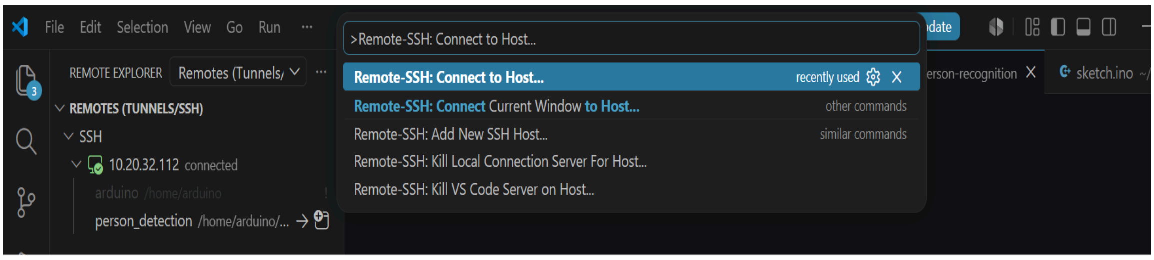
1. Install the Remote-SSH Extension

- Download and install **VS Code IDE** on your laptop.
- Open VS Code and open the Extensions view by clicking the square icon on the left sidebar (or press `Ctrl+Shift+X`).
- Search for **Remote - SSH** (published by Microsoft) and click **Install**.

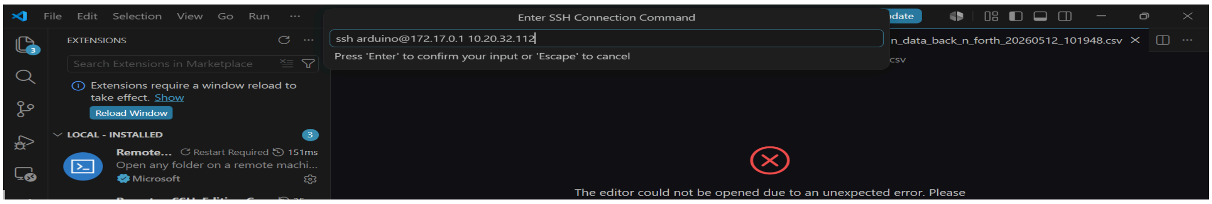


2. Establish the Wireless SSH Connection

- Press `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (Mac) to open the VS Code Command Palette.
- Type `Remote-SSH: Connect to Host...` and select it from the dropdown menu.

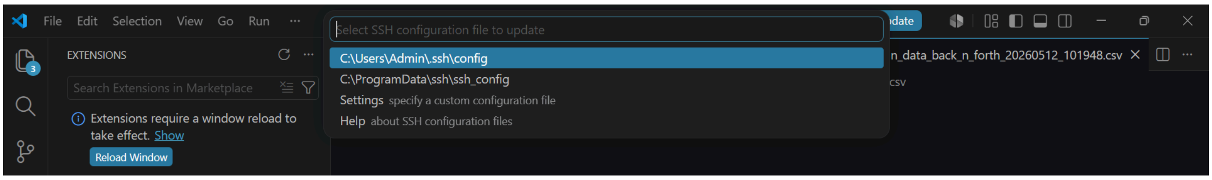


- Click **Add New SSH Host...** and type the connection string using your board's unique Wi-Fi IP address:



(Example: `ssh arduino@192.168.1.45`)

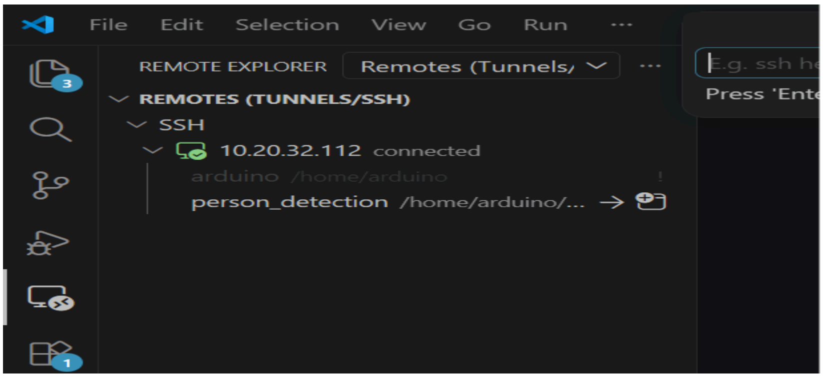
- Select your preferred local configuration file directory (the first user directory path option) to save this profile entry.



- A notification popup will appear in the bottom-right corner. Click **Connect** to launch a new, dedicated VS Code window.
- When prompted, select **Linux** as the target system platform and enter the system account password you created during the initial setup.

Step 3 — Managing Workspace Files Over Wi-Fi

Once the bottom-left corner of your VS Code window displays `SSH: <YOUR_BOARD_IP>`, your laptop is fully tunneled into the board over Wi-Fi.



- Click **Open Folder** inside the VS Code explorer panel.
- To access your custom App Lab applications, navigate to the storage path:

  Code snippet

  ```
  /home/arduino/ArduinoApps/
  ```

- Open your project folder to freely edit and save your configuration adjustments within `app.yaml` or adjust your Python logic inside `main.py`.
- **Deploying Machine Learning Binaries:** If you are running an Edge Impulse project, compile it for the Arduino UNO Q architecture target, download the executable `.eim` file bundle, and simply **drag-and-drop** the binary directly into the board's active model storage path via the VS Code sidebar:

  Code snippet

  ```
  /home/arduino/arduino-bricks/ei-models/
  ```